

RL-3

gaurav choudekar

October 2025

QUESTIONS

Develop a small code snippet to load the corresponding Gym environments and print out the respective state and action space. Develop a random agent to understand the reward function of the environment. Record your observations. (3 Points) (b) Implement the policy gradient algorithm with and without the reward to go and advantage normalization functionality. For the advantage normalization functionality, use a suitable

baseline function (Slide 36, Lecture 15 : Policy Gradient Methods). Test them on Cartpole- v0 and Lunarlander-v2 environments. The code developed must be able to run from

command line using options such as environment name, reward-to-go functionality (true or false), advantage normalization functionality (true or false), number of iterations, batch

size for policy gradient and other command line variables as you deem fit. Attach in your reports, a comparison of learning curve graphs (average return at each iteration) with and with

out advantage normalization and reward-to-go functionality. (8 Points)

(c) Study and report the impact of batch size on the policy gradient estimates on these environments.

LUNARLANDER AND CARTPOLE ENVIRONMENTS

The code implements policy gradient reinforcement learning algorithms for two environments: CartPole-v1 and LunarLander-v3 We modified the reward structure for training for better and earlier convergence. We later experimented with different configurations such as reward-to-go, advantage normalization, and batch sizes to observe their effects on learning performance.

Input preprocessing

For both the environments, we take policy gradient approaches like reward-to-go and advantage normalization. From command line we can choose to enable or disable these functionalities, we also take number of iterations and batch size as input arguments.

environment setup

We set up the environments using OpenAI Gym, initializing the policy and baseline networks, optimizers, and hyperparameters. Later we created a random agent to observe the environment dynamics like state transitions and rewards.

observations from random agent

CartPole-v1

- State Space: The state space is continuous and consists of four variables: cart position, cart velocity, pole angle, and pole angular velocity.
- Action Space: The action space is discrete with two possible actions: move the cart left or right.

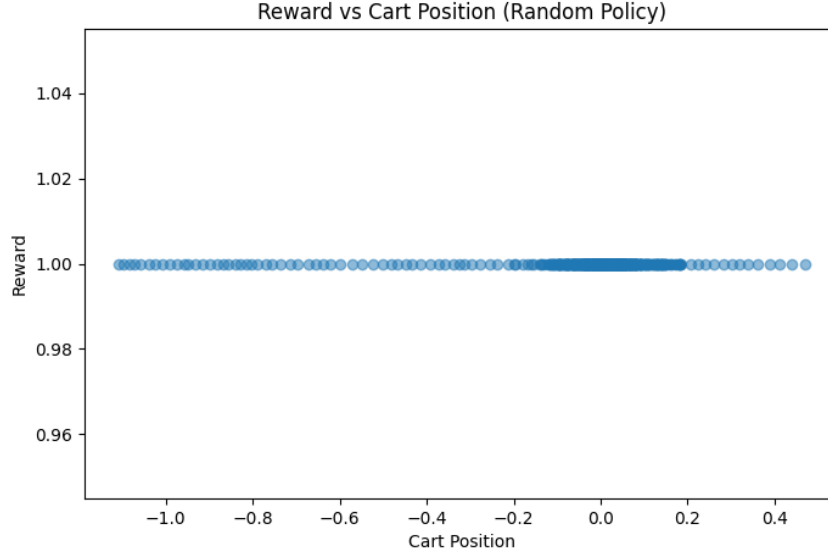


Figure 1: CartPole-v1: Random Agent State over Episodes

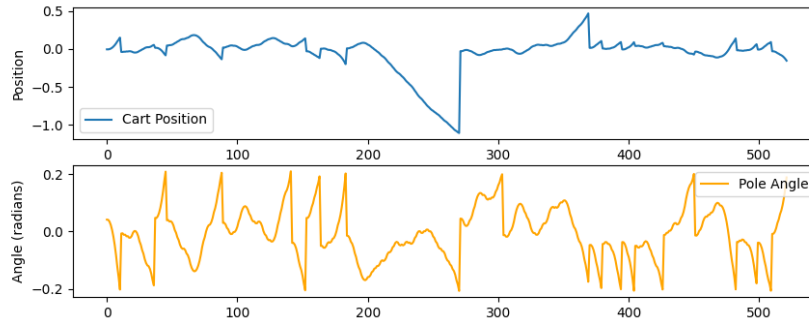


Figure 2: CartPole-v1: Random Agent Reward over actions

- Reward Function: The reward function gives a reward of +1 for every time step the pole remains upright.
- For every time step episode is running, we get a reward of 1.
- In our random simulation, we observed different numbers as observed reward per episode.
- Mostly our episodes ended because of the pole angle exceeding the threshold rather than cart position.
- We never observed the cart position exceeding the threshold in our random policy runs.
- This suggests that in a random policy, the pole angle is a more critical factor leading to episode termination than cart position.

LunarLander-v3

- State Space: The state space is continuous and consists of eight variables: x position, y position, x velocity, y velocity, lander angle, angular velocity, left leg contact, and right leg contact.
- Action Space: The action space is discrete with four possible actions: do nothing, fire left engine, fire main engine, fire right engine.

- Reward Function: The reward function provides rewards based on the lander's position, velocity, angle, and leg contact with the ground.
- In our random policy runs, we often observe negative rewards per episode.
- Mostly our episodes ended because of crash landing rather than successful landing.
- Mostly we observe -100 reward when the lander crashes, this shows that environment terminates episode with a penalty for crash
- we observe little positive rewards when the lander makes leg contact with ground, this shows that environment gives small bonus for leg contact

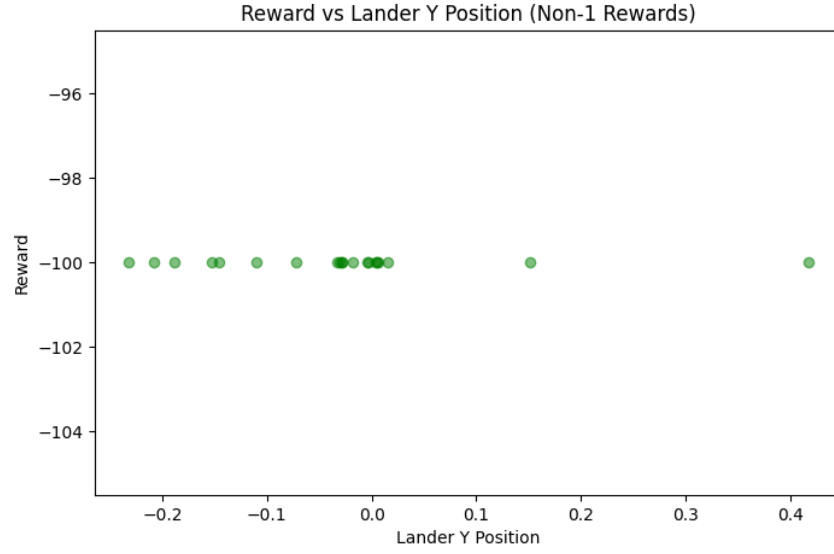


Figure 3: LunarLander-v3: Random Agent State over Episodes

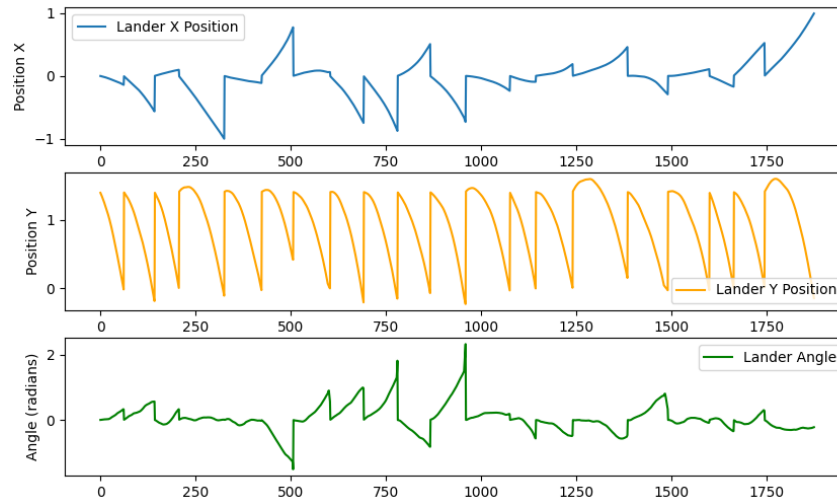


Figure 4: LunarLander-v3: Random Agent Reward over actions

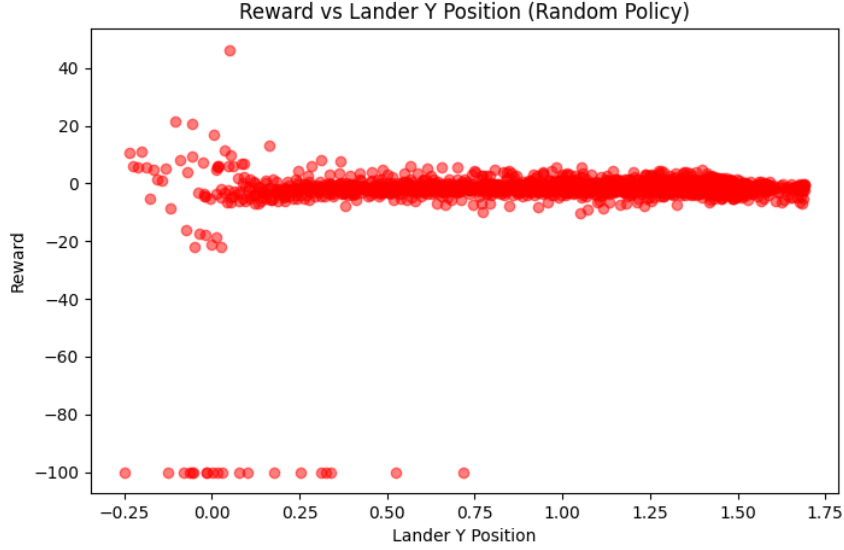


Figure 5: LunarLander-v3: Leg Contact Reward Bonus

Policy Gradient Implementation

We implemented the policy gradient algorithm with options for reward-to-go and advantage normalization, its algorithm is as follows:

- Collect trajectories by running the current policy in the environment until a batch of episodes is completed.
- For each trajectory, compute the rewards-to-go if the reward-to-go option is enabled.
- Estimate the advantages using the baseline network if advantage normalization is enabled.
- Update the policy network using the computed advantages and the collected states and actions.
- Update the baseline network to minimize the mean squared error between predicted values and actual returns.

Reward shaping

To improve learning efficiency, we modified the reward structure only for training the agent.

CartPole-v1

We gave a small negative reward based on absolute pole angle to keep the pole upright. We also gave a negative reward for crashing the pole to encourage the agent to avoid failure states.

LunarLander-v3

We introduced a shaping reward based on the lander's distance to the landing pad, its velocity, and angle to encourage stable landing. We also provided a bonus for leg contact with the ground and a penalty for crashing without leg contact.

Results and Observations

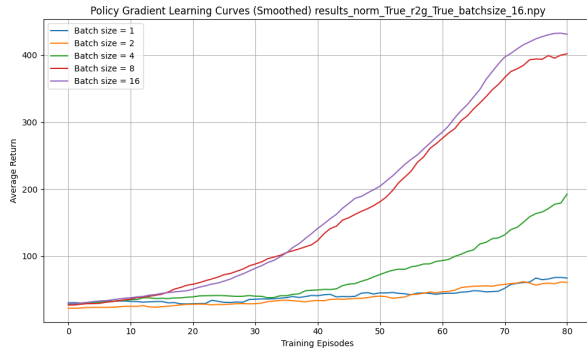
Results

Comparison of normalization and reward-to-go conditions

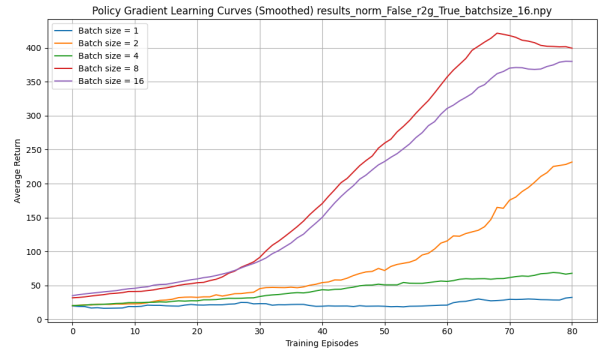
CartPole-v1

*Based on batch sizes Our parameters while testing on these 4 configurations for CartPole are :

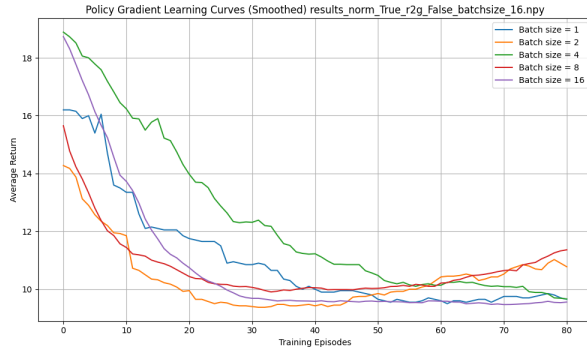
- Number of iterations - 80
- Discount factor 0.95
- Learning rate - 0.01
- Batch size - 16



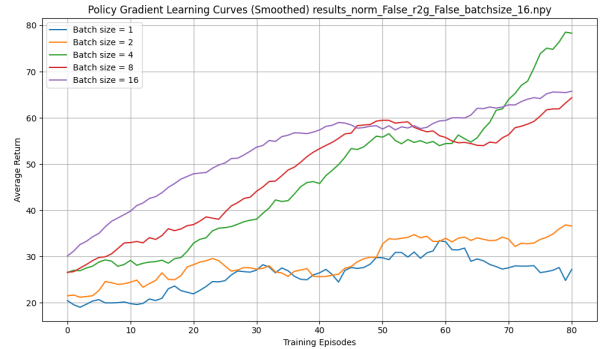
(a) Reward-to-go: on, Advantage norm: on



(b) Reward-to-go: on, Advantage norm: off



(c) Reward-to-go: off, Advantage norm: on



(d) Reward-to-go: off, Advantage norm: off

Figure 6: CartPole-v1: Average return per iteration comparing four configurations (reward-to-go and advantage normalization).

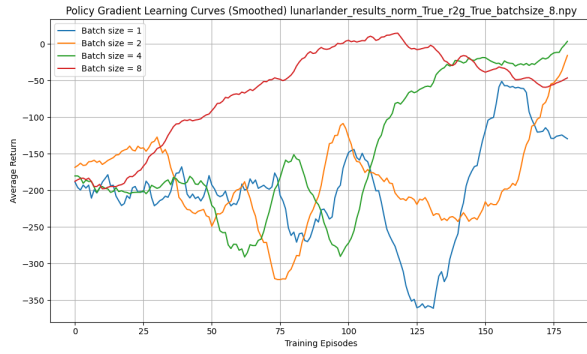
Important observations from these experiments:

- Larger batches provide better gradient estimates with lower variance
- Batch sizes 8 and 16 shows smoother learning curves compared to lower ones
- Batch size effectiveness works in combination with configuration for no norm R2G, higher batch sizes counter higher variance
- Batch sizes has minimal effect on configurations, all have unstable and falling learning curve

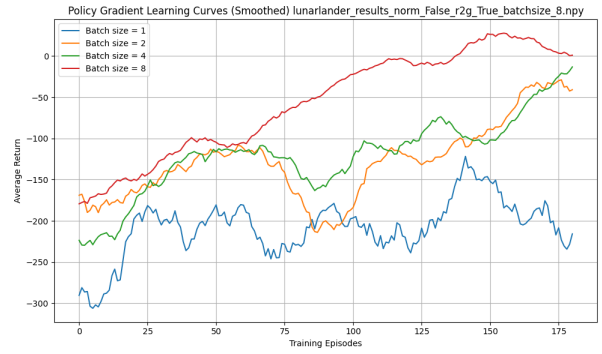
- Higher batch sizes tend to complete and start their new cycle/trend delayed
- Small batches learn quickly but less stables
- Large batches learn late but stable
- Even smaller batch sizes perform well for advantage norm and R2G condition both true

Our parameters while testing on these 4 configurations for LunarLander are :

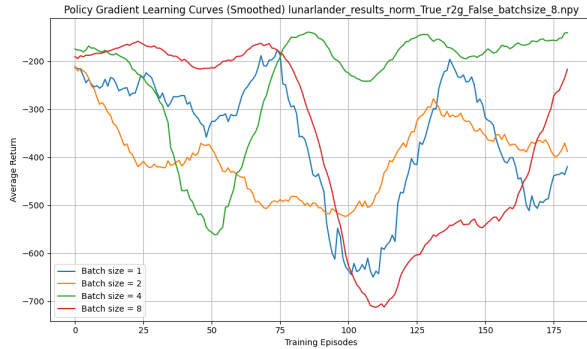
- Number of iterations - 200
- Discount factor 0.95
- Learning rate - 0.001
- Batch size - 8



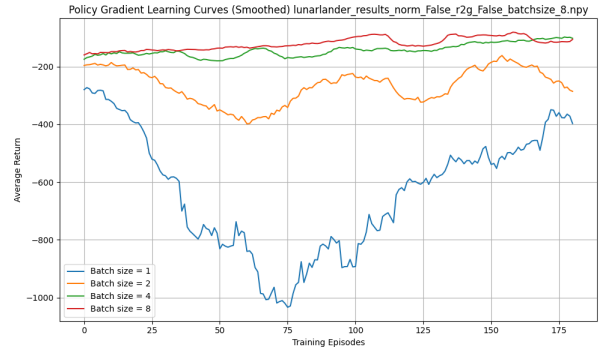
(a) Reward-to-go: on, Advantage norm: on



(b) Reward-to-go: on, Advantage norm: off



(c) Reward-to-go: off, Advantage norm: on



(d) Reward-to-go: off, Advantage norm: off

Figure 7: LunarLander-v3: Average return per iteration comparing four configurations (reward-to-go and advantage normalization).

Important observations from these experiments:

- Larger batches (4, 8) consistently yield smoother learning curves and higher final returns across all successful configurations.
- Smaller batches (1, 2) show high variance, instability, and often worse final performance
- For R2G without norm configuration is inherently unstable. Here, a batch size of 8 is essential to achieve decent performance

- The worst-performing learning curve across all charts is no norm, R2G with batch size 1. It combines high-variance updates (no norm) with a high-variance gradient estimator, leading to a failure to learn.
- R2G with norm is best performer
- Without R2G, all configurations fail to learn

*Comparison between advantage norm and reward calculation Our parameters while testing on these 4 configurations for CartPole are :

- Number of iterations - 100
- Discount factor 0.95
- Learning rate - 0.001
- Batch size - 8

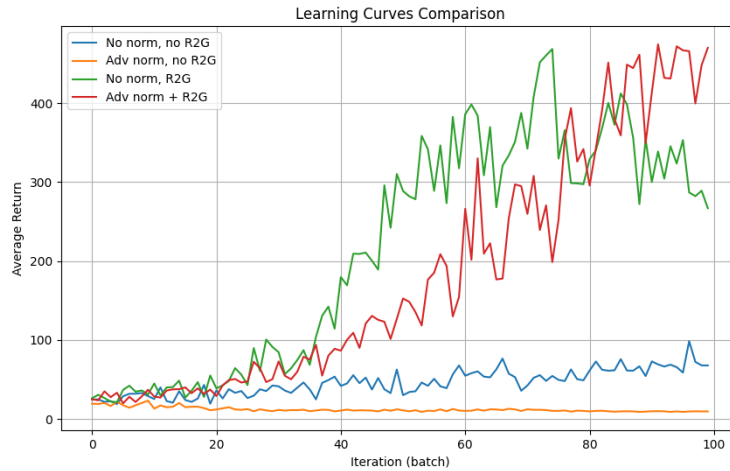


Figure 8: Comparisons between policy gradient conditions

Important observations from this experiment:

- Advantage norm with R2G is best performer, it has one side trend and never goes back
- R2G without advantage norm fails to perform high, changes its trend
- Normalization reduces gradient variance without sacrificing final performance, critical for stable convergence
- Advantage norm without R2G does not provide any good result, worse than no R2G and no advantage norms
- No R2G and no advantages norms almost gives random results

Our parameters while testing on these 4 configurations for CartPole are :

- Number of iterations - 200
- Discount factor 0.95
- Learning rate - 0.001

- Batch size - 8

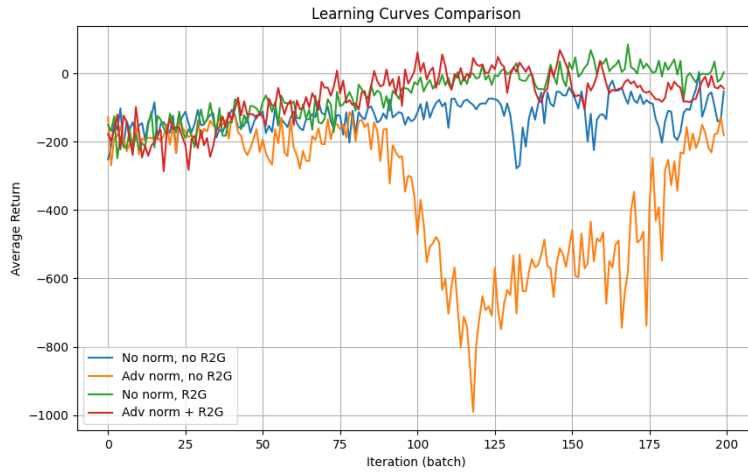


Figure 9: Comparisons between policy gradient conditions

Important observations from this experiment:

- The 2 configurations with R2G show clear and positive learning, which is not the case without it
- Adv norm + R2G learns faster and more consistently than No norm, R2G
- Normalization doesn't just stabilize learning; it actively makes it more sample-efficient
- Without both configurations, agent shows no signs of learning, remains almost random throughout
- Even with stable updates, learning from the wrong credit assignment signal is futile, making it perform worse than a noisier but more correct signal.

MOUNTAINCAR AND PONG ENVIRONMENTS

This section describes our experiments on the **MountainCar-v0** and **Pong-v4** environments. Unlike Cart-Pole and LunarLander, these two tasks are more challenging: MountainCar requires the agent to exploit environmental physics to reach the goal, while Pong involves vision-based learning using pixel input.

MountainCar-v0

The agent must drive a car up a steep hill, but the engine is not powerful enough to climb directly. The solution involves building momentum by first moving left and then right.

State and Action Space

- **State space:** Continuous with 2 variables — position (x) and velocity ($\dot{x}$).
- **Action space:** Discrete with 3 actions — push left, no push, push right.

Random agent

We used random agent to explore its movement and reward, we used it for 20 episodes(4000 steps) and observed this graph:

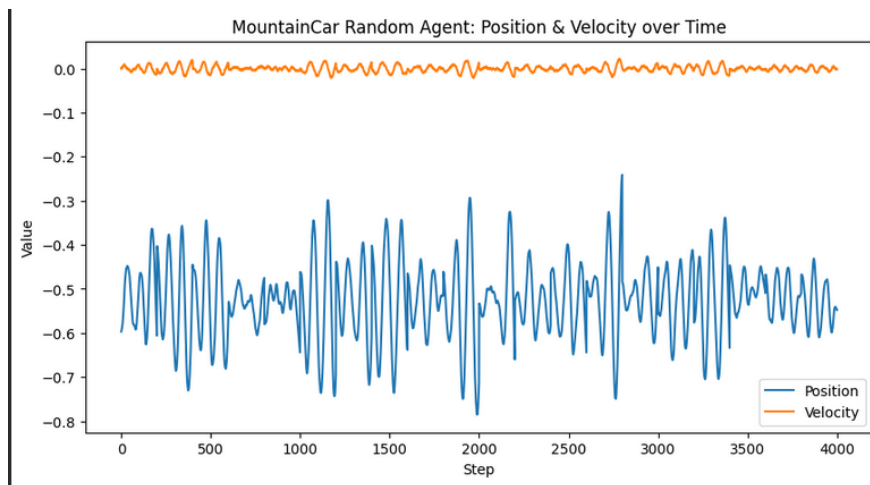


Figure 10: Random agent:Mountaincar

Observations from these random agent are:

- The position fluctuates mostly between approximately -0.8 and -0.3, never reaching the goal at the right side of the mountain
- The movement appears oscillatory, showing repeated small back-and-forth motions, typical of a random agent failing to gain enough momentum to climb the hill.
- The agent is “stuck” in the valley, unable to reach the goal due to lack of coordinated acceleration
- Due to this we never see position 0.5 and a non zero reward, everytime we see -1 as shown above

Reward Function and Shaping

We introduced our own reward shaping, used only for training not for evaluation purpose. It adds extra feedback to guide learning, here it rewards moving right, gaining speed, and reaching higher positions, while also encouraging early leftward movement for momentum. This makes training faster and smoother than using the sparse default -1 reward.

Algorithm

We used a Double DQN setup with a prioritized replay buffer (priority based on position). To study the impact of replay buffer size, we trained agents with four buffer sizes: {1000, 5000, 10000, 50000}. Huber loss was used for stability, and target networks were updated periodically.

Learning curve

Parameters used are:

- Buffer size = 10000
- Learning rate = 0.99
- Number of episodes = 1000

- batch size = 128

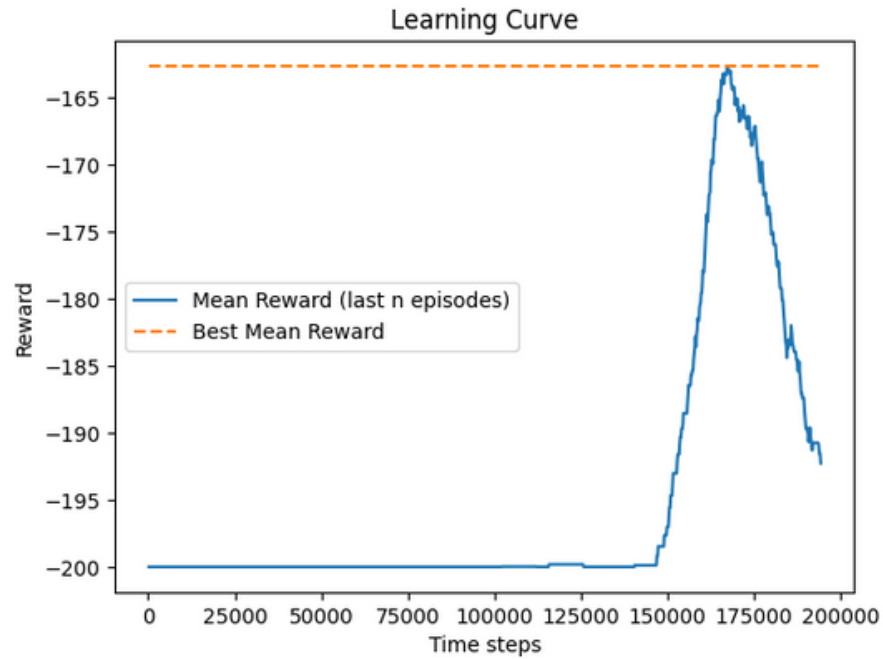


Figure 11: MountainCar-v0: Mean reward vs training steps

Observations from these experiment:

- The mean reward stays around -200 for many time steps, showing the agent struggled to learn an effective policy early on.
- Around 140k–170k steps, the curve sharply rises, indicating the agent finally discovered a better strategy to reach the goal more efficiently.
- After reaching peak, they start to unlearn due to high learning rate, or due to over exploration, this causes instability in policy updates
- The agent demonstrates learning progress but lacks long-term stability, suggesting potential benefit from better tuning or more stable learning algorithms.
- Proper/large number are always necessary during testing, else we would have considered optimal parameters near 175k steps

Different batch sizes

Parameters used are:

- Number of episodes = 1000
- Batch sizes = 1000, 5000, 50000, 100000

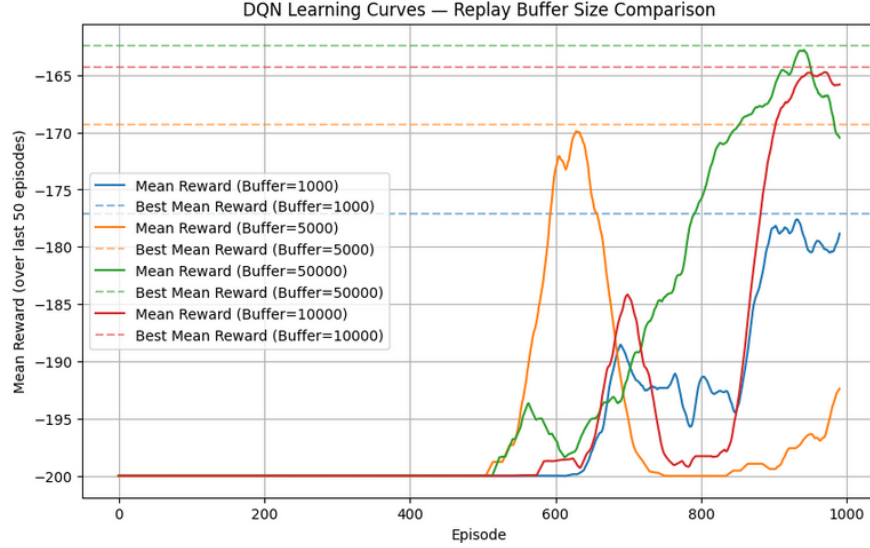


Figure 12: MountainCar-v0: Mean reward vs training steps for different batch sizes sizes.

Observations from these experiment:

- The agent with larger replay buffers (10,000 and 50,000) achieves smoother and higher rewards compared to smaller buffers.
- Small buffer sizes starts learning fast but their performance degrades later due to high variance, due to over fitting
- The 50,000-buffer achieves the highest and most stable mean reward , showing that a sufficiently large buffer enhances sample diversity and long-term performance.
- Increasing replay buffer size leads to better stability and convergence, though possibly at the cost of slower initial learning.
- Reversals are delayed for larger buffer sizes

Pong-v4

The **Pong-v4** environment is a visual task from the Atari suite, where the agent controls the paddle on the right side of the screen to hit the ball back to the opponent.

State and Action Space

- **State space:** Raw pixel frames ($210 \times 160 \times 3$), preprocessed to grayscale and resized to 84×84 .
- **Action space:** 6 discrete actions (e.g., NOOP, FIRE, UP, DOWN, RIGHTFIRE, LEFTFIRE).

0.0.1 Random Agent

We used random agent to know about reward system of environment

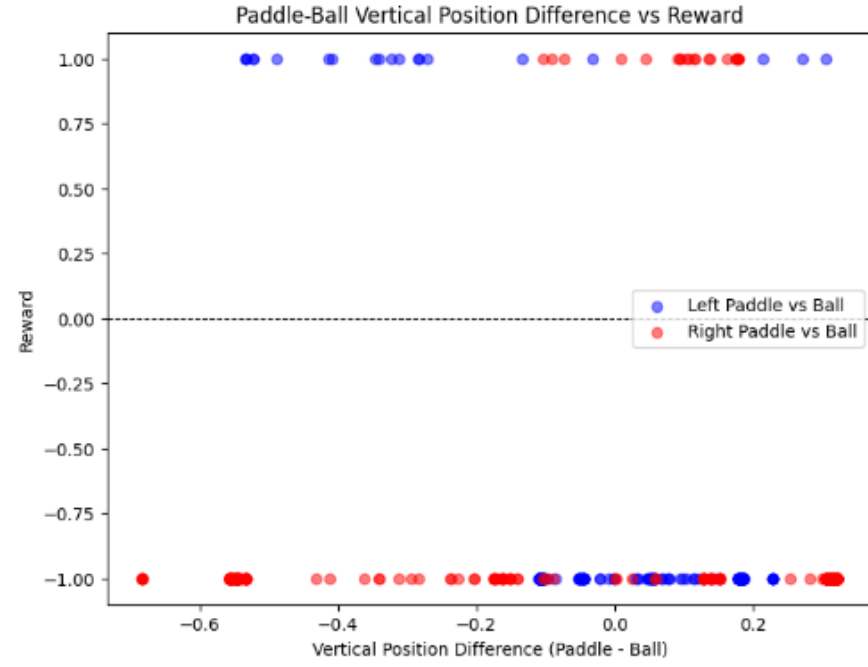


Figure 13: Pong: reward vs vertical distances of paddles

Observations from these experiment:

- Out of 25k steps, only 440 were non-zero rewards, summing positive rewards of agents was single digit
- This shows random agent is not efficient to play pong game
- The left paddle (opponent) consistently performs better, maintaining more frequent -1 rewards
- The random agent's vertical position difference with the ball is widely scattered, showing no learned coordination or timing in paddle movement.
- The opponent's actions are more aligned with the ball's trajectory
- For reward -1, more blue dots are visible near 0, while for +1 more red are visible near 0
- This shows they have better chances to earn points when they align themselves with ball, not aligning gives points to opponent

Frame Preprocessing

Each RGB frame is converted to grayscale, cropped, horizontally flipped, and resized to focus on the play area:

- Converts to grayscale.
- Crops to remove scoreboards and unused borders.
- Resizes to 84×84 for computational efficiency.
- Uses frame differencing to capture motion information.

Model and Training

We used a convolutional DQN with 3 convolutional layers followed by fully connected layers. Training used the Adam optimizer and Huber loss. Target networks were updated every 5000 steps.

Parameters during training:

- Replay buffer size: 10000.
- Batch size: 32.
- Discount factor: 0.99.

To enhance interpretability, we implemented a simple ball-and-paddle detection system that detects positions using intensity thresholds and highlights contact points between ball and paddle.

Reward Function

We give a slight positive reward shaping when we detect ball is hit by our agent, while we give bigger reward(+8) when we score a point

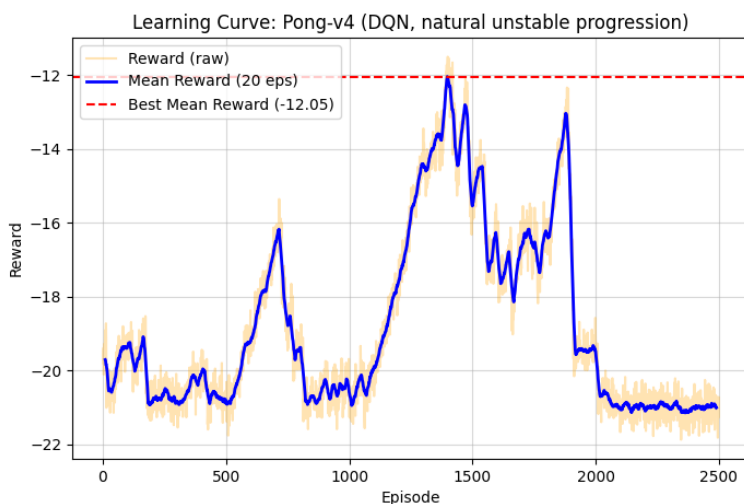


Figure 14: Pong: learning curve

Observations from these experiment:

- The agent initially performs poorly (average reward -21), typical for early random exploration in Pong.
- Gradual improvement occurs around episodes 500–1500, showing the agent begins to learn ball–paddle coordination and scoring strategies
- The training shows instability and oscillations, indicating the agent forgets previously learned strategies or overfits to certain states.
- A few sharp reward drops suggest sharp forgetting or instability in Q-value updates during training
- after certain point agent could not maintain and recover its earlier progress

using policy gradient

```
RL.py --env_name LunarLander-v3 --num_iterations 1000 --reward_to_go --baseline --normalize_advantages --discount_factor 0.95 --learning_rate 0.001 --batch_size 4
```